

Software Testing

Student Study Edition · Golden Master design · 11 structured lessons with original course visuals, examples, activities, and study aids.

01 · Testing Foundations

02 · Who Tests? Testing vs Debugging

03 · Functional Testing & Testing Levels

04 · Nonfunctional Testing & Documentation

05 · Testing Lifecycle & Defects

06 · Equivalence Partitioning

TOPIC 12 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Testing Foundations	3
02	Who Tests? Testing vs Debugging	5
03	Functional Testing & Testing Levels	7
04	Nonfunctional Testing & Documentation	9
05	Testing Lifecycle & Defects	11
06	Equivalence Partitioning	13
07	Boundary Value Analysis	15
08	Decision Table Testing	17
09	Test Planning, Test Cases & Defect Reporting	19
10	Concepts You Must Distinguish	21
11	Choose the Test from the Problem	24

TOPIC 12 · LESSON 01 · 1 OF 11 LESSONS

TOPIC 12 · SOFTWARE TESTING · 01

Testing Foundations

Turn requirements into evidence. Software testing executes the system deliberately to reveal failures, check expected behavior, and build confidence that the software is ready for use.

PRACTICE LAB · TOPIC 12

Mission 12 — Turn Requirements Into Test Evidence

Lead a 14-stage release review for CarePass in Jeddah. Distinguish testing levels, design functional and nonfunctional evidence, apply Equivalence Partitioning, Boundary Value Analysis and Decision Table Testing, then build reproducible test cases and defect reports.

CLO 17

Describe and distinguish among the different types and levels of testing: unit, integration, system, and acceptance.

THE TESTING MINDSET

Do not ask “Did we test it?” Ask “What evidence do we have?”

Every useful test connects a requirement or risk to a deliberate input, an expected result, an observed result, and a decision.

INPUT → TEST → EVIDENCE

Why do we test?

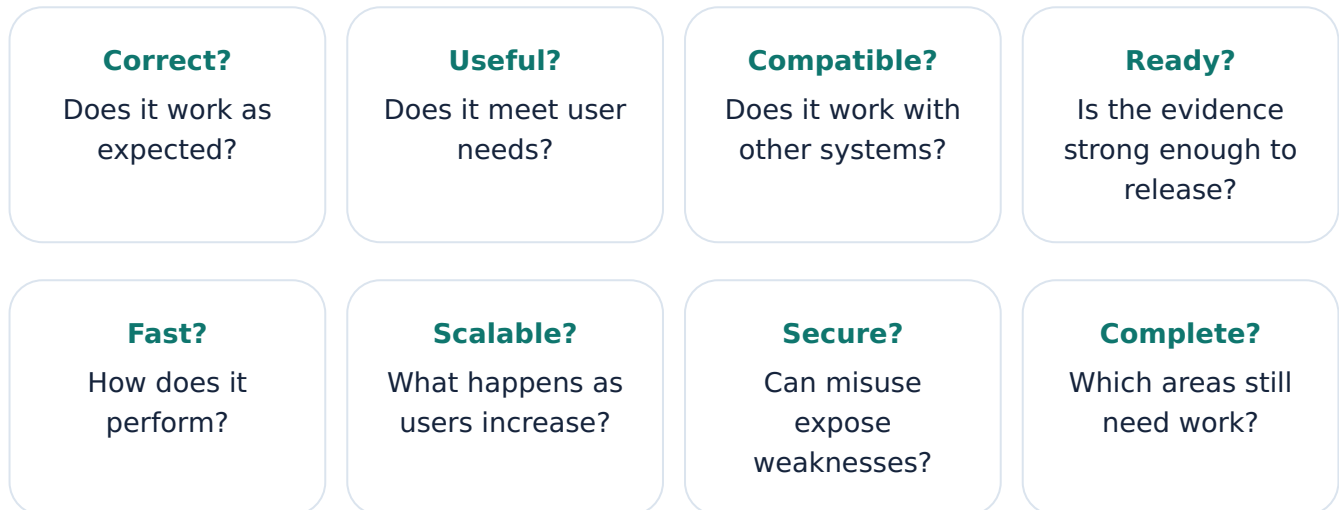
**Demonstrate satisfaction**

Show that functional and non-functional requirements are met under selected conditions.

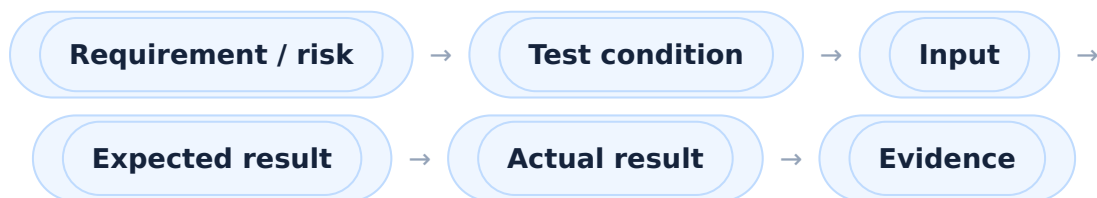
**Detect problems**

Expose errors, missing requirements, gaps, and undesired behavior before users discover them.

Testing answers release questions



A test is a small experiment



Important: A passed test provides evidence for the specific situation tested. It does not prove that the software contains no defects.

TOPIC 12 · LESSON 02 · 2 OF 11 LESSONS

02 · PEOPLE, EXECUTION & VISIBILITY

Who Tests? Testing vs Debugging

Testing is a team activity. Separate who participates, how tests are executed, and how much the tester knows about the internals.

Who participates?

**Software Tester**

Evaluates behavior against requirements and reports defects.

**Developer**

Tests units and investigates failures in code.

**Project Lead / Manager**

Uses quality evidence to manage readiness and risk.

**End User**

Provides acceptance evidence from the intended-use perspective.

Testing ≠ Debugging

Testing

Execute software to identify a failure, bug, error, defect, gap, or unexpected behavior.

Question: **What failed?**

≠

Debugging

Identify, isolate, and fix the cause of the observed problem.

Question: **Why did it fail, and how do we fix it?**

Two different axes

Execution style: Manual ↔ Automation

Manual testing is performed by a person using scenarios/test cases. Automation uses scripts or tools to rerun checks quickly and repeatedly.

Knowledge: Black ↔ Grey ↔ White Box

This describes how much the tester knows about the application's internal logic and structure.

Black Box

Inputs + outputs

Grey Box

Limited internal

White Box

Logic + structure

Internal code knowledge

No internal knowledge
requiredknowledge
Partial visibility**Quick diagnosis**

A developer examines branches inside a method after a test fails. The test exposed the failure; examining and correcting the branches is **debugging**, using internal/white-box knowledge.

Do not mix the axes: a black-box test can be manual or automated. Manual/automation tells you how; black/grey/white tells you what internal knowledge is used.

TOPIC 12 · LESSON 03 · 3 OF 11 LESSONS

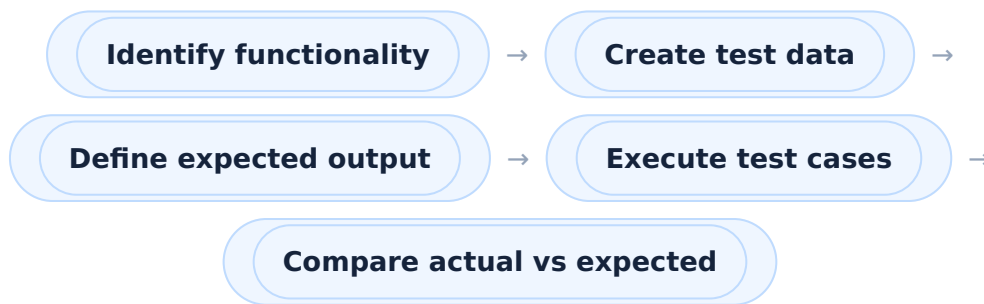
03 · WHAT IS TESTED VS WHERE IT IS TESTED

Functional Testing & Testing Levels

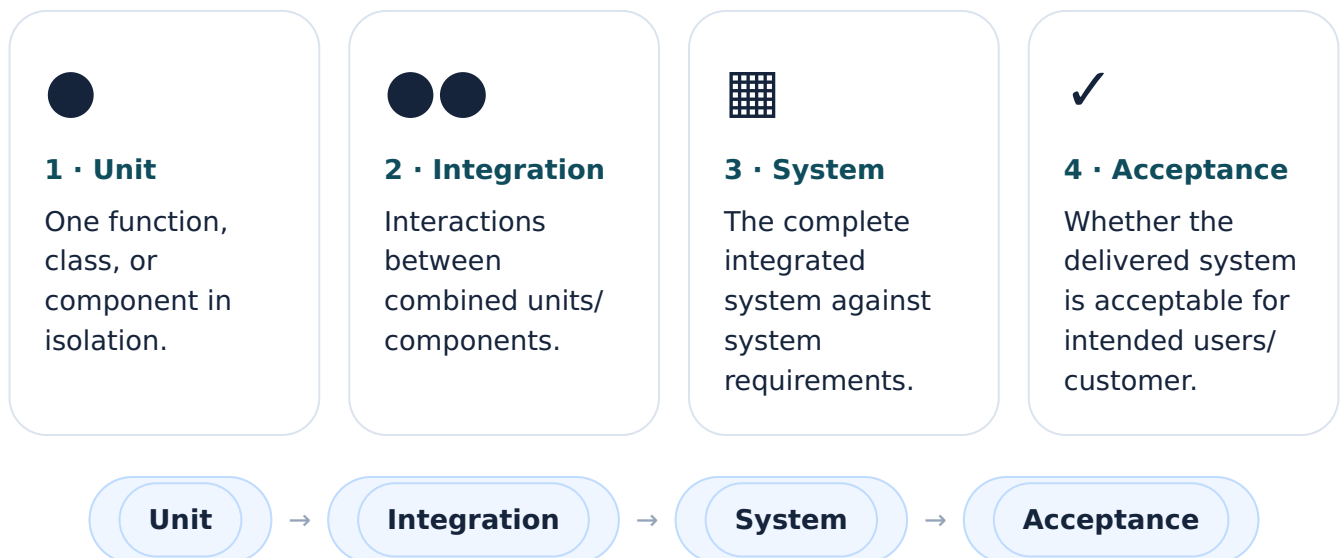
Keep two ideas separate: the quality being evaluated and the scope of software under test.

Functional testing asks: “Does it do the required job?”

Functional testing is based on software specifications. Supply inputs, observe outputs, and compare the actual behavior with the functionality the system is intended to provide.



Testing levels: zoom out through the system



One scenario across all four levels

System	Unit	Integration	System / Acceptance
Course Registration	Test prerequisite-check logic.	Test RegistrationService with Student and Section data sources.	Run the complete registration workflow, then validate it against

			registrar/student expectations.
--	--	--	---------------------------------

CLO connection: When asked to distinguish testing levels, identify the **scope under test**: one unit, interacting units, whole system, or delivered system from the customer/user perspective.

TOPIC 12 · LESSON 04 · 4 OF 11 LESSONS

04 · QUALITY ATTRIBUTES & TEST EVIDENCE

Nonfunctional Testing & Documentation

Correct functionality is not enough. Software must also provide evidence about performance, resilience, security, portability, and traceability.

Performance: four questions

Speed

How fast is the response?

Capacity

How much work can it handle?

Stability

Does behavior remain reliable over time?

Scalability

What happens as demand grows?

Potential bottlenecks include network delay, client-side processing, database transactions, server load balancing, and data rendering.

Load vs Stress

Load Testing

Observe behavior under normal and peak expected loads and large input volumes.

“Can it handle what we expect?”

VS

Stress Testing

Push beyond expected limits or reduce resources to create abnormal conditions.

“What happens when expectations are exceeded?”

Security testing: protect the system and its data

Security properties

Confidentiality · Integrity · Availability · Authentication · Authorization · Non-repudiation

Input & session risks

Input validation · Injection flaws · SQL injection · Session management

Application risks

Cross-site scripting · Buffer overflows · Directory traversal · known/unknown vulnerabilities

Portability & testing documentation

Portability testing checks whether software can be moved or run across computers/platforms. Documentation makes the testing work visible, repeatable, estimable, and traceable.

Test Plan

Scope, approach, resources, schedule.

Test Scenario

A behavior or situation to evaluate.

Test Case

Inputs, steps, expected result, actual result.

Traceability Matrix

Connect requirements to test evidence.

Scenario: Thousands of students registering at 8:00 AM is a **load** question. Intentionally exceeding expected concurrency or removing resources is a **stress** question.

TOPIC 12 · LESSON 05 · 5 OF 11 LESSONS

05 · FROM PLANNING TO DEFECT CLOSURE

Testing Lifecycle & Defects

Testing is not a single phase after coding. It begins with planning and requirements, continues through execution and defect handling, and finishes with evidence about release readiness.

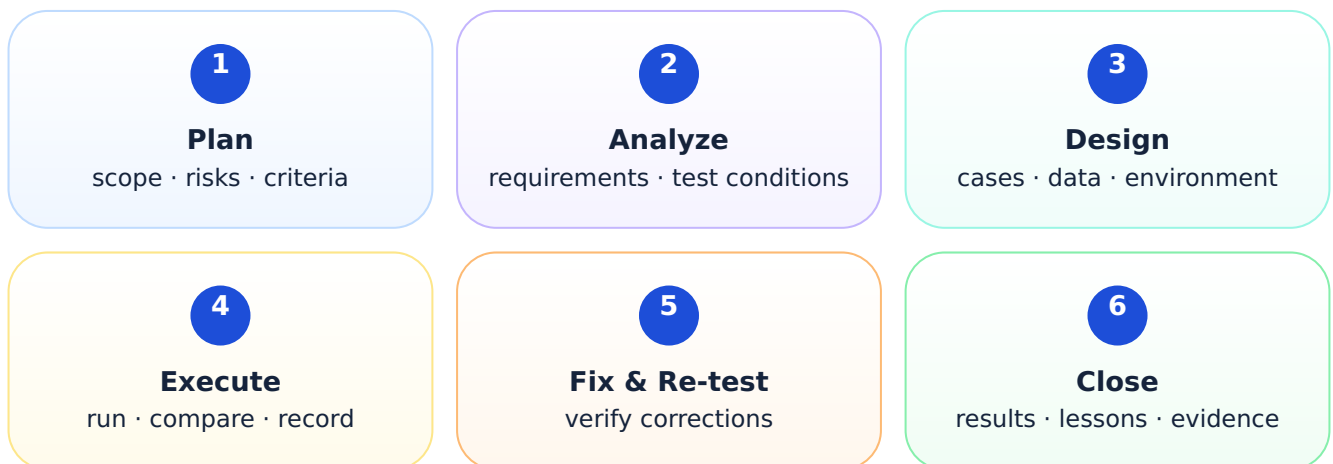
TEST EARLY · LEARN EARLY

The later a defect survives, the more expensive its impact can become.

Early review of requirements and design can prevent defects before they reach code, integration, system testing, or production.

INPUT → TEST → EVIDENCE

A lightweight Software Testing Life Cycle

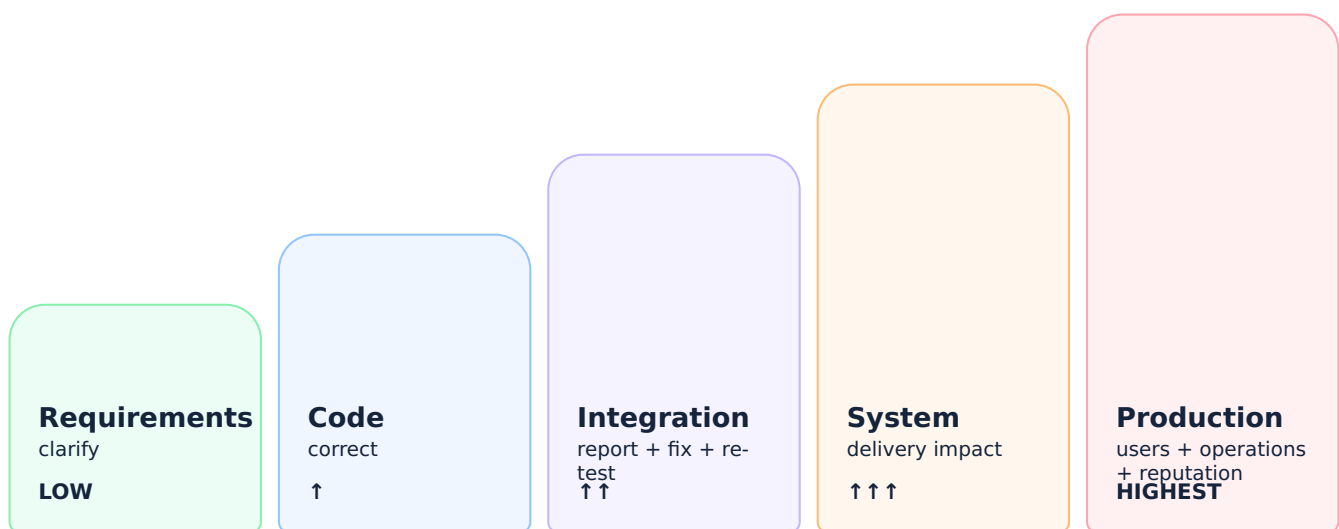


Defect lifecycle



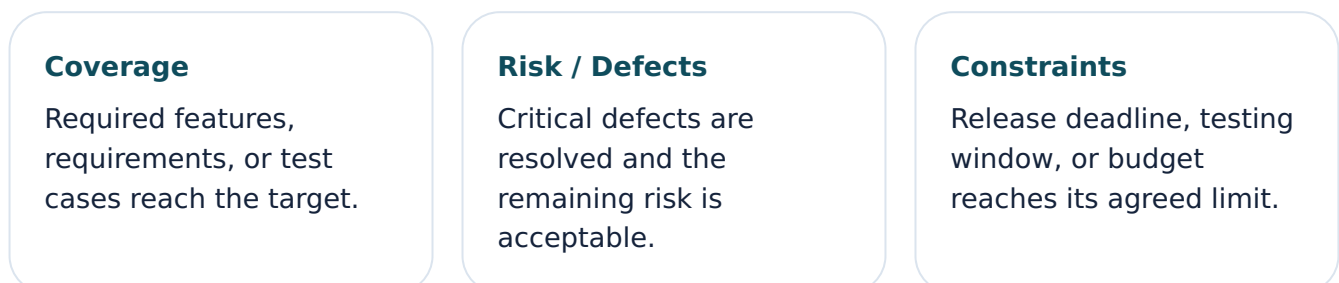
If the fix fails verification, the defect can be **re-opened** and return to the development loop.

Why early detection matters



When do we stop testing?

Not when we have “proved there are no bugs.” Testing is reduced or stopped when agreed exit evidence is sufficient.



Key connection: Testing finds evidence of problems. Defect tracking makes those problems visible until they are fixed, re-tested, and closed.

TOPIC 12 · LESSON 06 · 6 OF 11 LESSONS

06 · BLACK-BOX TEST DESIGN

Equivalence Partitioning

Stop testing random values. Divide a large input domain into behavioral regions, then choose representatives from each region.

CORE IDEA

Many inputs → few meaningful partitions → representative tests

Inputs within one partition are assumed to produce equivalent behavior, so we deliberately sample the partitions rather than every possible value.

INPUT → TEST → EVIDENCE

Example 1 · Date must be 1-31

Invalid

≤ 0
... -2, -1, 0

Valid

1-31
1, 2, ... 31

Invalid

≥ 32
32, 33, ...

Example 2 · Username must be 6-10 characters

Invalid

0-5 characters

Valid

6-10 characters

Invalid

11+ characters

Representative set: choose one value from each region, for example 3, 8, and 12. The goal is partition coverage, not exhaustive testing.

Example 3 · Age 18-80 except 60-65



This example matters because valid/invalid regions do not have to form one continuous interval.

More study examples

Exam score · 0-100

Invalid <0 · Valid 0-100 ·
Invalid >100 .
Representatives: -5, 75,
105.

Quantity · 1-5 items

Invalid ≤ 0 · Valid 1-5 ·
Invalid ≥ 6 .
Representatives: 0, 3, 7.

Shipping weight

0-5 kg = standard · >5 -
20 kg = heavy · >20 kg
= unsupported. Each
behavior is a distinct
partition.

Try it · classify a username length

Enter a username length.

Exam workflow: 1) identify the rule → 2) write all valid and invalid partitions → 3) select representative values. Do not jump straight to test values.

TOPIC 12 · LESSON 07 · 7 OF 11 LESSONS

07 · BLACK-BOX TEST DESIGN

Boundary Value Analysis

After Equivalence Partitioning finds the regions, Boundary Value Analysis targets the edges where software behavior is most likely to change.

EP → BVA

Find the regions. Then challenge their borders.

The lecture's rule is to test both sides of each boundary because behavior may differ immediately across the edge.

INPUT → TEST → EVIDENCE

Date rule · 1 to 31

0just below lower
boundary**1**at valid lower
boundary**31**at valid upper
boundary**32**just above upper
boundary

Username rule · 6 to 10 characters

5

below lower

6

valid lower

10

valid upper

11

above upper

Floating point · 0.2 to 0.8, one decimal place

0.1

below lower

0.2

valid lower

0.8

valid upper

0.9

above upper

More study examples

Exam score · 0-100

Quantity · 1-5

Timeout · 10-30 sec

Use -1, 0, 100, 101 to challenge the two edges.

Use 0, 1, 5, 6. The middle value 3 is valid, but it does not challenge an edge.

Use 9, 10, 30, 31. With decimals allowed, also consider precision around the boundary.

EP vs BVA

Equivalence Partitioning

Which behavioral regions exist?

Think regions.



Boundary Value Analysis

Where do those regions meet?

Think edges.

Practice · course capacity is 1-40

Boundary-focused values: **0, 1, 40, 41**. Values such as 17 and 28 are inside the same valid partition and do not challenge a boundary.

Use EP and BVA together: EP gives the map; BVA tells you where to zoom in.

TOPIC 12 · LESSON 08 · 8 OF 11 LESSONS

08 · BLACK-BOX TEST DESIGN

Decision Table Testing

When behavior depends on combinations of conditions, turn the logic into explicit rules so missing combinations become visible.

CONDITIONS → RULES → ACTIONS

Complicated logic becomes a table you can test systematically.

Decision tables document condition combinations and the expected action for each rule.

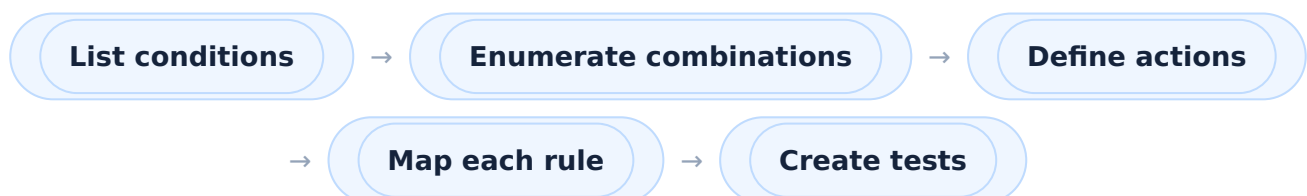
INPUT → TEST → EVIDENCE

Email + password example

Condition / Rule	R1	R2	R3	R4
Email present?	F	T	F	T
Password present?	F	F	T	T
Expected action	Enter email	Enter password	Enter email	Login processed

Two independent Boolean conditions produce $2^2 = 4$ combinations.

How to build a decision table



Applied example · Course Add Request

Condition / Rule	R1	R2	R3	R4
------------------	----	----	----	----

Prerequisite satisfied?	F	F	T	T
Seat available?	F	T	F	T
Action	Reject: prerequisite	Reject: prerequisite	Reject: full	Add course

More study examples

Online discount

Conditions: member?
coupon valid? order $\geq$ 200? Start with up to 2^3
= 8 Boolean combinations.

ATM withdrawal

Conditions: card valid?
PIN correct? balance sufficient? The action changes by the combination, so a table exposes missing rules.

Parcel pickup

Conditions: code valid?
parcel ready? locker online? Actions may be open locker, retry, or show service error.

Diagnose the technique

If a discount depends on **member?**, **coupon?**, and **order $\geq$ 200?**, use a decision table because the outcome depends on combinations. With three independent Boolean conditions, start by considering $2^3 = 8$ combinations.

Recognition cue: If the requirement contains several IF/AND/OR conditions that combine to produce different actions, think **Decision Table**.

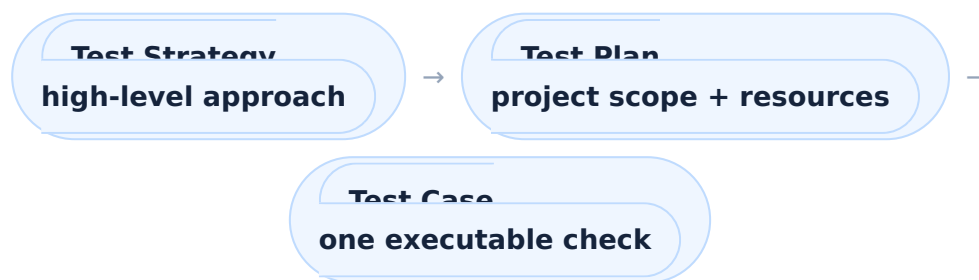
TOPIC 12 · LESSON 09 · 9 OF 11 LESSONS

09 · FROM REQUIREMENT TO REPRODUCIBLE EVIDENCE

Test Planning, Test Cases & Defect Reporting

Plan the testing work, turn requirements into repeatable test cases, and report defects clearly enough that another person can reproduce and fix them.

Strategy → Plan → Case



Test Strategy asks

Which methods and levels? Which risks first? What environment, metrics, automation approach, and standards?

Test Plan asks

What is in/out of scope? Who tests? When? With what environment/data? What are pass/fail and exit criteria?

Anatomy of a useful test case

ID	Setup / Input	Steps	Expected Evidence
TC-LOGIN-01	Valid student account	Enter email + password → Sign in	Dashboard opens · PASS/FAIL recorded

Example · Positive

Valid course code + available section → section details displayed.

Example · Negative

Invalid course code → clear validation error; no registration attempted.

Example · Security

Unauthorized user attempts admin URL → access denied and event logged.

Manual or automated?

Manual

+

Automation

Flexible and useful for exploration, new UI behavior, and situations that need human judgment.

Example: usability observation with first-time students.

Best when a stable test is repeated frequently and consistency/speed provide value.

Example: run the login regression suite after every build.

Automation decision: Favor repeatable, important, stable tests. Automation does not replace test design; it automates execution of selected tests.

A good defect report is a reproducible story

BUG-142 Registration accepts a full section

HIGH

ENVIRONMENT Chrome · macOS · build 2.4	LOCATION Course Registration
STEPS TO REPRODUCE Open full section → click Add → confirm	EXPECTED Registration rejected: "Section full"
ACTUAL Registration record is created	STATUS Open · Assigned

Severity ≠ Priority

Severity

How strongly does the defect affect the software or user?

Impact Example: payment charged twice → very severe.

≠

Priority

How urgently should the team fix it?

Order of action Example: typo on tomorrow's launch screen may be low severity but high priority.

Communication rule: Finding a defect is only half the job. A concise report with environment, steps, expected result, and actual result makes the defect actionable.

TOPIC 12 · LESSON 10 · 10 OF 11 LESSONS

10 · STUDY ESSENTIALS

Concepts You Must Distinguish

These concepts appear across the two course sources and are easy to confuse in exams. Learn them by contrast and by scenario, not by memorizing isolated definitions.

Quality Assurance vs Quality Control

Quality Assurance (QA)

Focuses on preventing defects by improving and following processes, standards, templates, reviews, and readiness activities.

Process-oriented · Prevent

VS

Quality Control (QC)

Focuses on evaluating the product and identifying whether it meets required quality and performance expectations.

Product-oriented · Detect

Example · Which is QA and which is QC?

Defining a mandatory code-review checklist is **QA**. Executing tests on the completed feature to find failures is **QC/testing**.

Verification vs Validation

Verification

Review requirements, designs, code, plans, or specifications to check that the product is being built according to defined artifacts.

“Are we building it correctly?”

VS

Validation

Execute/evaluate the software to check that the delivered behavior satisfies intended requirements and user needs.

“Are we building the right product?”

Example · Course Registration

Reviewing the registration requirement for completeness is **verification**. Running the system and confirming that eligible students can register correctly is **validation**.

Re-testing vs Regression Testing

Re-testing

Run the failed scenario again after a fix to confirm that the original defect is actually corrected.

Did we fix this defect?

+

Regression Testing

Run relevant existing tests after a change to detect new problems introduced elsewhere.

Did the fix break something else?

Example · Login fix

A password-reset bug is fixed. Running the same reset scenario is **re-testing**. Re-running login, account lockout, session, and profile-access tests is **regression testing**.

Exploratory, Usability & Compatibility Testing

Exploratory Testing

The tester learns and adapts while testing instead of following only a fixed script.

Example: freely explore checkout after noticing unusual cart behavior.

Usability Testing

Observe whether intended users can complete tasks effectively and understand the interface.

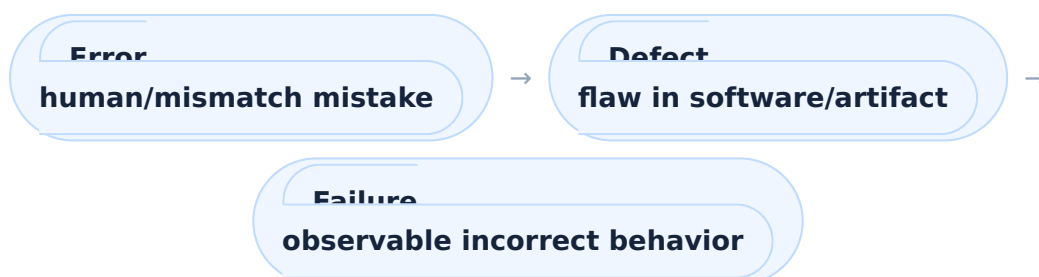
Example: ask a new student to register for a course without help.

Compatibility / Configuration

Check behavior across browsers, operating systems, devices, or configurations.

Example: Safari vs Chrome, mobile vs desktop.

Testing terms: error, defect, failure



Example · Discount calculation

A developer writes the wrong formula (**error**). The wrong formula remains in the code (**defect**). A customer receives a wrong total during checkout (**failure**).

Study rule: When two terms look similar, ask what they focus on: process or product, artifact or execution, original defect or side effects, scripted or adaptive testing.

TOPIC 12 · LESSON 11 · 11 OF 11 LESSONS

11 · REVIEW & PRACTICE

Choose the Test from the Problem

Use this page as your study checkpoint. First identify the testing question, then select the level, method, technique, document, or follow-up activity that fits.

Complete one-minute map

Scope?

Unit →
Integration →
System →
Acceptance

Visibility?

Black → Grey →
White Box

Input design?

Regions → EP
Edges → BVA

Combined logic?

Conditions +
actions →
Decision Table

Quality attribute?

Performance · Load · Stress · Security ·
Portability · Usability · Compatibility

After a defect?

Fix → Re-test original problem →
Regression test affected areas

Quality activity?

QA prevents through process · QC/testing
detects product problems

Evidence?

Strategy → Plan → Test Case → Defect
Report → Traceability

Diagnose the testing problem

1 · Username must be 6–10 characters.
EP: 0–5 invalid, 6–10 valid, 11+ invalid.
BVA: 5, 6, 10, 11.

2 · Tester sees only inputs and outputs.
Black-box testing.

3 · Developer investigates branches
inside code.
White-box knowledge. If locating and
fixing a failure's cause, that work is
debugging.

4 · Checkout depends on member?
coupon? stock?
Decision Table Testing because
behavior depends on combinations of
conditions.

5 · 5,000 users are the expected peak.
Load testing. Going deliberately beyond
the expected limit is a stress test.

6 · Test communication between two
services.
Integration testing.

7 · Customer decides whether delivered system is acceptable.

Acceptance testing.

8 · Requirement must be linked to evidence.

Use a **Traceability Matrix** to connect requirements to tests.

9 · A fixed password-reset defect is tested again.

Testing the exact failed scenario is **re-testing**. Re-running related login/session tests is **regression testing**.

10 · Team reviews requirements before coding.

Verification. Running the implemented system against user needs is validation.

11 · Tester freely explores checkout and adapts based on discoveries.

Exploratory testing.

12 · App works in Chrome but fails in Safari.

Compatibility/configuration testing.

Worked study set · Smart Course Registration

A course accepts 1–40 students. Registration requires prerequisite completion and an available seat. Thousands of students may access the system at opening time, and the same system must work in multiple browsers.

EP

Capacity partitions: ≤ 0 invalid · 1–40 valid · ≥ 41 invalid.

BVA

Values: 0, 1, 40, 41.

Decision Table

Prerequisite × seat availability determines add/reject action.

Integration

RegistrationService ↔ Student/Section data sources.

Load / Stress

Expected opening traffic vs deliberately exceeding capacity.

Compatibility

Repeat critical workflow on Chrome, Safari, and mobile browser.

Mini exam-style questions

A field accepts age 18–65. Give EP and BVA.

EP: <18 invalid, 18–65 valid, >65 invalid. **BVA:** 17, 18, 65, 66.

A payment fix is confirmed, but cart totals now fail. What happened?

The original fix may pass **re-testing**, but the cart failure is a **regression** that should be detected by regression testing.

A typo is visible on tomorrow's launch page. Low severity or low priority?

Likely **low severity** but possibly **high priority** because it is highly visible and release is imminent.

Final study mindset: Do not memorize a list of testing names. For every scenario ask: What is the scope? What risk or behavior matters? What input structure exists? What evidence is needed? What should happen after a defect is found?